```
import os
import cv2
import numpy as np
import urllib.request

image_path = "https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcS9z

image = None

if image_path.startswith("http://") or image_path.startswith("https://"
    try:
        print("Downloading image from URL...")
        req = urllib.request.Request(image_path, headers={"User-Agent":
        response = urllib.request.urlopen(req, timeout=10)
        image_bytes = np.asarray(bytearray(response.read()), dtype=np.u
        image = cv2.imdecode(image_bytes, cv2.IMREAD_GRAYSCALE)
        if image is None:
            print("Could not decode the downloaded file as an image.")
        else:
            print("Loaded image from URL.")
    except Exception as e:
        print("Could not download image from URL:", e)

elif os.path.exists(image_path):
    print("Loading image from local path...")
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        print("Could not read the local image file.")
    else:
        print("Loaded image from local path.")

else:
    print("Invalid image path or URL.")

from google.colab.patches import cv2_imshow
if image is not None:
    cv2_imshow(image)
```

```
Downloading image from URL...
Loaded image from URL.
```

```
if image is None:
    print("No input image found. Creating a sample test image instead.")
    image = np.zeros((300, 300), dtype=np.uint8)
    cv2.rectangle(image, (50, 50), (150, 150), 200, -1)
    cv2.line(image, (0, 250), (300, 250), 180, 4)
```

```
gaussian_blur = cv2.GaussianBlur(image, (5, 5), sigmaX=0)
```

```
median_filtered = cv2.medianBlur(image, 5)
```

```
average_filtered = cv2.blur(image, (5, 5))
```

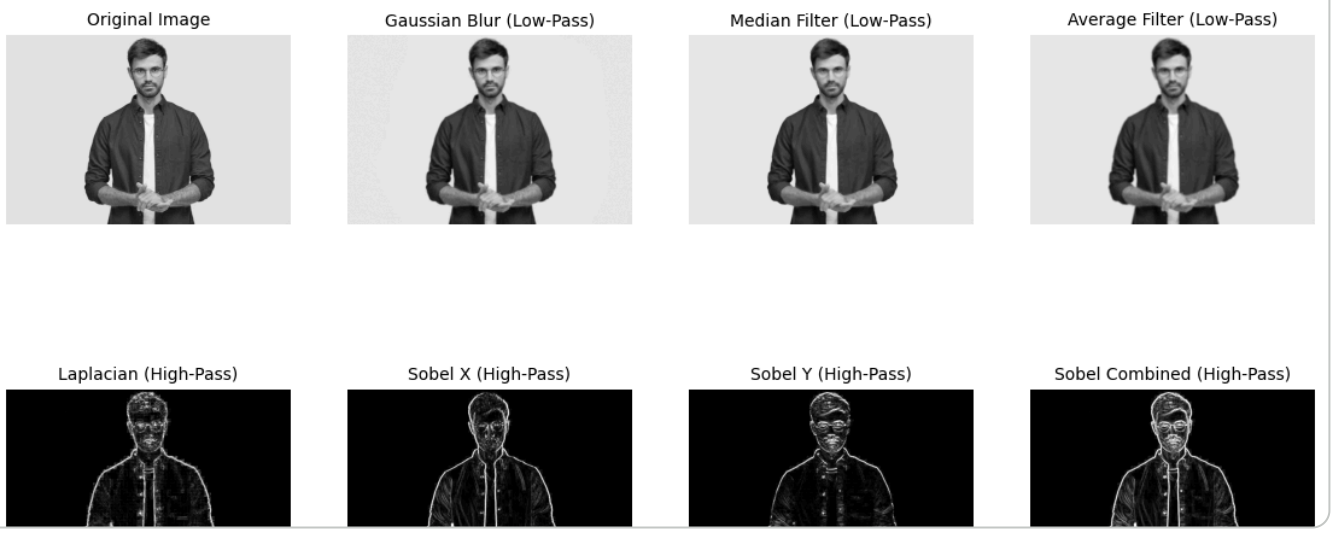
```
laplacian = cv2.Laplacian(image, cv2.CV_64F, ksize=3)
laplacian_display = cv2.convertScaleAbs(laplacian)
```

```
sobel_x = cv2.Sobel(image, cv2.CV_64F, dx=1, dy=0, ksize=3)
sobel_y = cv2.Sobel(image, cv2.CV_64F, dx=0, dy=1, ksize=3)
sobel_x_display = cv2.convertScaleAbs(sobel_x)
sobel_y_display = cv2.convertScaleAbs(sobel_y)
sobel_combined = cv2.convertScaleAbs(cv2.magnitude(sobel_x, sobel_y))
```

```
titles = [
```

```
images_to_show = [
    image,
    gaussian_blur,
    median_filtered,
    average_filtered,
    laplacian_display,
    sobel_x_display,
    sobel_y_display,
    sobel_combined,
]
```

```
plt.figure(figsize=(14, 7))
for i in range(len(images_to_show)):
    plt.subplot(2, 4, i + 1)
    plt.imshow(images_to_show[i], cmap="gray")
    plt.title(titles[i], fontsize=10)
    plt.axis("off")
```




```
plt.tight_layout()
plt.savefig("filtering_results.png", dpi=150)
print("Saved comparison figure as filtering_results.png")
plt.show()
```

```
Saved comparison figure as filtering_results.png
<Figure size 640x480 with 0 Axes>
```

```
:("\n--- Observations ---")
:("Low-Pass Filters (Gaussian, Median, Average): reduce noise but blur e
:("Median filter works best on salt-and-pepper noise (dots), keeps edges
:("High-Pass Filters (Laplacian, Sobel): highlight edges but amplify noi
:("Sobel X detects vertical edges; Sobel Y detects horizontal edges.")
```

```
--- Observations ---
Low-Pass Filters (Gaussian, Median, Average): reduce noise but blur edges.
Median filter works best on salt-and-pepper noise (dots), keeps edges sharper than Aver
High-Pass Filters (Laplacian, Sobel): highlight edges but amplify noise.
Sobel X detects vertical edges; Sobel Y detects horizontal edges.
```